

Intermediate (watch the model choose between two tools) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — a plain counting loop

```
# tool_selection_practice.py – Intermediate section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])

counts = {"get_weather": 0, "get_forecast": 0, "none": 0}

for i in range(5):
    response = model_with_tools.invoke("what's the weather like")
    if not response.tool_calls:
        counts["none"] += 1
    else:
        picked = response.tool_calls[0]["name"]
        counts[picked] = counts.get(picked, 0) + 1

print(counts)
```

Expected output (an example split — yours may vary run to run):

```
{'get_weather': 3, 'get_forecast': 2, 'none': 0}
```

This version works and gives you a real split. It doesn't print per-run detail, which makes it harder to spot patterns (like "it always picked X on the exact same wording") — worth adding once you're looking closely.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a reusable trial function with per-run logging

```
# tool_selection_practice.py – Intermediate section
from collections import Counter
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather, get_forecast])

def run_selection_trial(prompt: str, runs: int = 5) -> Counter:
    model_with_tools = get_model_with_tools()
    tally: Counter = Counter()

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
        print(f"Run {run_number}: picked {picked}")
        tally[picked] += 1

    return tally

if __name__ == "__main__":
```

```
result = run_selection_trial("what's the weather like")
print(f"\nFinal split: {dict(result)}")
```

Expected output (example):

```
Run 1: picked get_weather
Run 2: picked get_forecast
Run 3: picked get_weather
Run 4: picked get_weather
Run 5: picked get_forecast
```

```
Final split: {'get_weather': 3, 'get_forecast': 2}
```

Difference from Basic: printing each run's pick as it happens, not just the final tally, lets you actually look for a pattern (does the exact same prompt always pick the same tool, or does it genuinely vary?) — a detail the Basic version throws away. Using `Counter` instead of a hand-built dictionary also avoids needing a `.get(picked, 0)` fallback, since `Counter` handles unseen keys automatically.

Which one should you actually write? Intermediate's `run_selection_trial()` at 5 runs is enough for a quick, one-off look at whether two descriptions overlap — that's what this document's Build Task needs. Before writing down a real conclusion other people will act on, two hidden variables are worth checking: whether the *order* tools are registered in shifts the outcome (run the same trial with the tool list reversed), and whether 5 runs is even enough data to trust (a close split needs a much bigger sample before you treat it as signal, not noise). Neither is required for this exercise.